

Micro-controller Interfacing

RS-232

- Physical characteristics
 - Type
 - Full-duplex
 - Peer-to-peer
 - Connections/Wires/Pins
 - Tx: Transmit
 - Rx: Receive
 - Gnd
 - Connector
 - Old: DB-25
 - Standard: DB-9
 - Half Serial Cable: Only 3 wires connected -- Rx, Tx, Ground.
 - Full Serial Cable: 9 wires connected
 - Rx, Tx, Ground.
 - CTS, RTS -- Hand-shaking signals
 - Voltage/current levels
 - NRZ (Not Returning Zero)
 - CMOS Voltage levels
 - +3V to +25V -- Logic 0 -- Space
 - -3V to -25V -- Logic 1 -- Mark
 - TTL Voltage (0V-0 and 5V-1) ----> RS-232 Line Driver ----> CMOS Voltage levels
 - TTL <--> MAX-232 <--> CMOS
 - Bit-rate / Baud rate
 - Standard baud rates: 9600, 38400, 115200, ...
- Logical characteristics
 - Data frame

- Start bit --> Always 0 (space) -- 1 start bit
- Data bits --> 5 to 9 bits (Usually 8-bits) -- LSB Tx First
- Parity bit --> Even/Odd/1/0/No
 - Even/Odd -- configure of even or odd number of 1 bits.
 - Sticky 1/0 -- Parity bit always 1 or 0.
 - No -- Parity disabled i.e. no Parity bit added.
- Stop bit(s) --> Always 1 (mark) -- 1 or 2 stop bits
 - For lower baud rates, 1 stop bit is enough.
 - For higher baud rates, 2 stop bits are recommended.
- Error conditions
 - Parity error: Parity mismatched on receiver end (w.r.t. config).
 - Frame error: Stop bit is 0.
 - Read overrun: The uC is processing some data and not read the received data. The received data is overwritten by next received data.
 - Noise error: Detected in STM32 with oversampling.

I2C Protocol

Physical characteristics

- Type
 - Bus protocol (Master Slave)
 - Half duplex
- Wires/Connectivity
 - 2 wire protocol
 - SDA -- Serial Data
 - SCL -- Serial Clock
- Voltage levels
 - TTL voltage
- Frequency
 - 100 KHz -- Standard Mode
 - 400 KHz -- Fast Mode
 - 1 MHz -- Fast plus Mode

Logical characteristics

- I2C Device Modes
 - Master Transmitter
 - Master Receiver
 - Slave Transmitter
 - Slave Receiver
- Data Bit transfer
- Data frame
 - 8 Data Bits + ACK
 - Transmitter ---> 8 Data Bits + (ACK=1) ---> Receiver
 - Transmitter <--- Acknowledge (ACK=0) <--- Receiver
- Address frame
 - 7 I2C-Addr Bits + Read/Write' + ACK
 - Master ---> 7 I2C-Addr Bits + Read/Write' + (ACK=1) ---> Slave
 - Master <--- Acknowledge + (ACK=0) <--- Slave

Single byte data write

- START (M) + I2CSlaveAddr_WR (MT) + ACK (SR) + InternalAddress (MT) + ACK (SR) + DataByte (MT) + ACK (SR) + STOP (M)

Multi byte data write

- START (M) + I2CSlaveAddr_WR (MT) + ACK (SR) + InternalAddress (MT) + ACK (SR) + DataByte1 (MT) + ACK (SR) + ... + STOP (M)

Single byte read write

- START (M) + I2CSlaveAddr_WR (MT) + ACK (SR) + InternalAddress (MT) + ACK (SR) + REPEAT START (M) + I2CSlaveAddr_RD (MT) + ACK (SR) + DataByte1 (ST) + ACK (MR) + STOP (M)

Multi byte data read

- START (M) + I2CSlaveAddr_WR (MT) + ACK (SR) + InternalAddress (MT) + ACK (SR) + REPEAT START (M) + I2CSlaveAddr_RD (MT) + ACK (SR) + DataByte1 (ST) + ACK (MR) + ... + STOP (M)

Clock stretching

- If slave is busy in processing the received data, it pulls clock line (SCL) low. So effective line voltage is low and clock is stretched (disabled).
- Due to clock disabled, no I2C data is transferred.

Arbitration

- If two devices send start signal simultaneously (to become master), whichever device send low (0) bit first, wins the arbitration.
- The winning device continues master operation, while losing device fallback to slave mode.

Device Modes

- Master Transmitter
- Master Receiver
- Slave Transmitter
- Slave Receiver

I2C Device - State machine

- I2C device as a state machine.
 - Each I2C operation has standard success/failure codes.
 - Status code is of 5 bits.

Error conditions

- Bus error -- When START or STOP signal is received while communication is going on (Bus is busy). In master mode, it will keep owning the bus (will not release bus lines).
- Ack error -- After transmitting data/address, transmitter keeps data line high. The receiver after getting the data, pulls line low. This is ack to the transmitter. If no receiver devices, pulls line low will cause Ack error -- indicate that data is not received.

- Arbitration lost error
- Read overrun -- The next received data overwrites earlier received data before reading it into some regr.
- STM32 -- CRC Error -- CRC of transmitter and receiver mismatch.

SPI

Physical characteristics

- Type
 - Full-duplex protocol
 - Bus protocol
- Wires/Connectivity
 - 4 wire protocol
 - MOSI/SDO
 - MISO/SDI
 - SCLK/SCK
 - SS/CE
- Internal block diagram
 - Single shift register is used for full duplex communication
- Voltage levels
 - TTL voltage
 - 0 V = Logic 0
 - 3/5 V = Logic 1
- Frequency
 - High speed

Logical characteristics

- Data bit transfer
 - CPOL --> Clock Base
 - CPHA --> Leading/Trailing Edge
- Data frame

- Single byte transfer
 - Slave selection is responsibility of the master.
 - $SS=0 + MOSI \rightarrow 8 \text{ Data Bits} \rightarrow MISO + SS=1$
 - $SS=0 + MISO \leftarrow 8 \text{ Data Bits} \leftarrow MOSI + SS=1$
- Multi-byte write
 - Master sends internal address (1-byte or 2-byte) followed by Number of data bytes.
 - $SS=0 + Master \rightarrow Addr1, Addr2, Byte1, Byte2, \dots, Byten \rightarrow Slave + SS=1$
- Multi-byte read
 - Master sends internal address (1-byte or 2-byte) and then Slave send Number of data bytes.
 - $SS=0 + Master \rightarrow Addr1, Addr2 \rightarrow Slave + Slave \rightarrow Byte1, Byte2, \dots, Byten \rightarrow Master + SS=1$

Applications/Devices

- SD card
- Accelerometer
- EEPROM
- LCD Display
- 7-Segment Display
- RTC
- ...

LIS3DSH (datasheet)

- Introduction
- SPI interface
 - Multi-byte read operation
 - Multi-byte write operation
- Registers
 - CTRL4 -- Data Rate + Axis Enable
 - OUT_X_L & OUT_X_H, OUT_Y_L & OUT_Y_H, OUT_Z_L & OUT_Z_H
 - STATUS

LIS3DSH Library Code - Reference

- You may refer the article below. However it is not for LIS3DSH.
 - <https://ruturajn.hashnode.dev/interfacing-lis302dl-accelerometer-using-the-spi-protocol-with-the-stm32f407vg-disc1-development-kit>

Error conditions

- Read overrun: Before reading received data the next data arrives and overwrite the old one. Thus old data is lost.
- Write collision: Before transmitting current data, new data is overwritten on it. Thus last data is not transmitted correctly.
- Mode fault: When SPI device is in master mode and other SPI device force it to become slave, then Mode fault occurs.
- Slave abort: If SPI master is communicating with a SPI slave and that slave stop responding suddenly, then this error is raised.
- STM32 -- CRC Error: STM32 SPI has CRC calculate & checking hardware in SPI. If CRC is not matching, then this error is raised.

Daisy chaining

- SPI Master select the slave using Slave Select pin. For multiple slaves it (master device) use multiple GPIO pins.
- The number of pins can be reduced using Mux circuit. In this case 3 GPIO pins will be needed to select one of the max 8 slave devices from SPI bus.
- Daisy chaining enables using single GPIO pin of master device to enable all the slaves and transfer data to them.
- <https://www.maximintegrated.com/en/design/technical-documents/app-notes/3/3947.html>
- <https://www.best-microcontroller-projects.com/spi-interface.html>

3-wire SPI

- Tie DOUT and DIN pin together.

CAN

Core concepts

CAN (Controller Area Network), is a robust and widely used communication protocol in the realm of embedded systems, particularly in automotive and industrial applications. It was developed by Bosch in the mid-1980s to facilitate reliable communication between various electronic control units (ECUs) in vehicles.

- How CAN protocol works?

1. **Message-Based Communication:** CAN is a message-based protocol, meaning data is transmitted in discrete packets called frames. These frames consist of a header and payload, and they are transmitted over the CAN bus.
2. **Differential Signaling:** CAN uses differential signaling to enhance noise immunity. In this scheme, data is transmitted differentially between two wires: CAN_H and CAN_L. This helps mitigate the effects of electromagnetic interference (EMI) and noise, making CAN suitable for use in noisy industrial environments.
3. **Arbitration:** CAN employs a non-destructive bitwise arbitration mechanism to ensure that multiple nodes on the network can access the bus without causing data corruption. When multiple nodes attempt to transmit simultaneously, the one with the highest priority (determined by its identifier, or ID) wins the arbitration process and gains bus access.
4. **Error Detection and Handling:** CAN includes built-in error detection mechanisms to ensure data integrity. This includes cyclic redundancy check (CRC) for error detection and acknowledgement mechanisms for error handling. If an error is detected during transmission, the transmitting node can retry the transmission.
5. **Two Frame Formats:** CAN supports two frame formats: standard and extended. Standard frames use an 11-bit identifier, while extended frames use a 29-bit identifier. This allows for a larger number of unique message IDs, enabling more flexible and scalable communication networks.
6. **Masterless and Peer-to-Peer Communication:** CAN operates in a masterless fashion, meaning there is no central node controlling communication. Instead, all nodes have equal access to the bus, facilitating peer-to-peer communication and decentralized control.

Overall, CAN protocol offers reliable, deterministic, and efficient communication suitable for real-time embedded systems in automotive, industrial automation, medical devices, and other applications requiring robust communication networks. Its widespread adoption and proven track record make it a cornerstone technology in the field of embedded systems.

CAN Standard/Extended Frames

In the CAN (Controller Area Network) protocol, there are two main frame formats: standard and extended. These formats define how data is structured and transmitted over the CAN bus. Here's a breakdown of each:

1. Standard CAN Frame:

- **Identifier:** Standard CAN frames use an 11-bit identifier, which allows for up to 2^{11} (or 2048) unique message IDs. This identifier determines the priority of the message during arbitration on the bus.

- **Data Length Code (DLC):** The DLC field specifies the number of bytes of data contained in the frame. It can range from 0 to 8 bytes.
- **Data:** This field contains the actual data payload of the message, with a maximum length of 8 bytes.
- **CRC (Cyclic Redundancy Check):** The CRC field is used for error detection. It contains a checksum calculated from the data and is used by the receiving node to verify the integrity of the received message.
- **Acknowledgement (ACK):** The ACK field indicates whether the message was successfully received by other nodes on the bus.
- **End of Frame (EOF):** The EOF field marks the end of the frame transmission.

2. Extended CAN Frame:

- **Identifier:** Extended CAN frames use a 29-bit identifier, significantly expanding the number of unique message IDs to 2^{29} (over 500 million). This allows for a much larger address space, accommodating a wider range of devices and applications.
- **Data Length Code (DLC):** Similar to standard frames, the DLC field specifies the number of bytes of data contained in the frame, ranging from 0 to 8 bytes.
- **Data:** Just like in standard frames, this field contains the actual data payload of the message, with a maximum length of 8 bytes.
- **CRC (Cyclic Redundancy Check):** The CRC field is used for error detection, calculating a checksum from the data to verify its integrity.
- **ACK:** Similar to standard frames, the ACK field indicates whether the message was successfully received by other nodes on the bus.
- **EOF:** The End of Frame field marks the end of the frame transmission.

The key difference between standard and extended CAN frames lies in the length of the identifier field, with extended frames allowing for much larger address spaces. This distinction enables extended frames to be used in applications requiring a greater number of unique message IDs, such as complex automotive networks or industrial automation systems.

CAN Bus Electrical Characteristics

The CAN (Controller Area Network) bus is a robust communication protocol used in various applications, including automotive, industrial automation, and more. Understanding its electrical characteristics is crucial for designing and implementing reliable communication networks. Here are the key electrical characteristics of the CAN bus:

1. **Differential Signaling:** CAN uses differential signaling, which means that it transmits data over two wires: CAN_H and CAN_L. The voltage difference between these two wires determines the state of the signal. This differential signaling helps improve noise immunity and allows for reliable communication in noisy environments.
2. **Voltage Levels:**

- In the recessive (logic 1) state, both CAN_H and CAN_L are set to 2.5 V.
- CAN_H and CAN_L voltages pulled by 1V so that updated voltages are 1.5 to 3.5 volts, when transmitting a dominant (logic 0) state.
- The voltage difference between CAN_H and CAN_L during dominant and recessive states is typically around 2 to 0 volts.

3. Termination Resistors:

- Termination resistors are used at both ends of the CAN bus to match the characteristic impedance of the bus (usually 120 ohms).
- These resistors help minimize signal reflections and ensure signal integrity by properly terminating the transmission line.
- Signal reflections can cause signal distortion, interference, and data corruption, leading to communication errors.
- Termination resistors help maintain signal integrity by absorbing reflections and preventing them from interfering with the transmitted signals.

4. Bit Timing:

- CAN uses a bit-wise arbitration mechanism to determine message priority on the bus.
- The bit timing parameters, such as bit rate, synchronization, and sampling points, are crucial for proper communication.
- CAN controllers typically support various bit rates, commonly ranging from a few kilobits per second (Kbps) to several megabits per second (Mbps), depending on the application requirements.

5. Physical Layer Standards:

- CAN physical layer standards, such as ISO 11898-2 for high-speed CAN (up to 1 Mbps) and ISO 11898-3 for low-speed/fault-tolerant CAN (up to 125 Kbps), define the electrical characteristics and requirements for CAN communication networks.
- These standards specify parameters such as voltage levels, termination resistor values, and physical connector specifications to ensure interoperability between different CAN devices.

6. Noise Immunity:

- CAN is designed to provide high noise immunity, allowing for reliable communication in harsh electromagnetic environments.
- Differential signaling, along with proper termination and shielding techniques, helps mitigate the effects of electromagnetic interference (EMI) and noise on the bus.

CAN Error States/Frames

CAN (Controller Area Network) error frames are special frames transmitted on the CAN bus to signal various error conditions that may occur during communication. These error frames help maintain the integrity and reliability of the CAN bus by providing mechanisms for error detection and recovery. Here are the main types of CAN error frames:

1. Error Active State (ERR_ACTIVE):

- This state indicates that no errors have been detected on the CAN bus, and the network is operating normally.
- In this state, the CAN controller sends six dominant bits after detecting an error. When the limit is below TEC (Transmit Error Counter) <127 and REC (Receive Error Counter) <127, an Error Active node will transmit Active Error Flags when it detects errors.
- In the error active state, nodes on the bus can continue transmitting and receiving messages without any issues.

2. Error Passive State (ERR_PASSIVE):

- When errors are detected/occurred on the CAN bus above a threshold, a node may transition to the error passive state.
- When any one of the two Error Counters raises above 127 (TEC > 127 and REC > 127), the node will enter a state known as Error Passive.
- In this state, the node is still able to transmit messages, but it observes stricter error handling rules. For example, it may delay transmission attempts or limit the number of retransmissions.

3. Bus Off State (BUS_OFF):

- If the number of transmission errors exceeds a certain threshold within a specified time period, a node may enter the bus off state.
- When the Transmit Error Counter raises above 255 (TEC > 255 and REC > 255), the node will enter the Bus Off state. It happens when the CAN controller fails or at times of extreme accumulations of errors.
- In this case, the CAN controller disconnects itself from the CAN bus. As a result, that particular CAN node will get switched off from the CAN network; Resulting in no communication at all (until it undergoes a recovery process).

4. Error Frames:

- Error frames are special messages transmitted on the bus to indicate the occurrence of errors.
- There are two types of error frames: active error frames and passive error frames.
 - **Active Error Frame:** This frame is transmitted when a node detects a bus error while in the error active state. It consists of six dominant bits (logical 0s) followed by an error delimiter bit.
 - **Passive Error Frame:** This frame is transmitted when a node detects a bus error while in the error passive state. It consists of six recessive bits (logical 1s) followed by an error delimiter bit. Receiving nodes don't globalize such errors (do not transmit again).

5. Error Detection and Handling:

- CAN nodes continuously monitor the bus for errors, such as bit errors, frame errors, and stuff errors.
- When an error is detected, the node takes appropriate actions based on its error handling rules, such as transitioning to a different error state or transmitting error frames.
- When error is detected, the transmitted frame is destroyed.
- Transmitting node resend the frame.

CAN Error Recovery

What Triggers Bus-Off?

A node enters the **Bus-Off** state when its **Transmit Error Counter (TEC)** exceeds **255**. This typically indicates severe or persistent transmission errors (e.g., hardware faults, wiring issues, or noise).

Recovery Process

The CAN protocol specifies an **automatic recovery mechanism**:

Step 1: Node Isolation

- The node is **disconnected** from the CAN bus (stops transmitting/receiving messages) to avoid disrupting the network.

Step 2: Automatic Recovery Attempt

- The CAN controller (hardware) waits for **128 occurrences of 11 consecutive recessive (1) bits** on the bus.
 - **Purpose:** Ensures the bus is idle and stable before rejoining.
 - **Why 11 recessive bits?:** Matches the CAN protocol's bit-stuffing rule (no more than 5 consecutive identical bits), guaranteeing a clean bus state.

Step 3: Error Counter Reset

- After detecting the required 128 x 11 recessive bits:

- The **TEC** is reset to **0**.
- The **Receive Error Counter (REC)** is set to **0** (or a value ≤ 127 , depending on the controller).
- The node transitions back to the **Error-Active** state (normal operation).

Step 4: Rejoining the Network

- The node resumes normal operation:
 - Participates in arbitration.
 - Transmits messages.
 - Monitors for errors.

Example Scenario

1. **Faulty Transmission:** A sensor node experiences a short circuit, causing repeated transmission errors. Its TEC exceeds 255, entering Bus-Off.
2. **Isolation:** The sensor stops communicating, allowing other nodes to function normally.
3. **Automatic Recovery:**
 - The CAN controller monitors the bus for 128 x 11 recessive bits (indicating prolonged stability).
 - Once detected, TEC/REC are reset, and the node rejoins the network.
4. **Normal Operation:** The sensor resumes sending data.

Real-World Use Case

In an automotive system:

- A faulty ECU enters Bus-Off due to a wiring issue.
- After the wiring is repaired, the ECU automatically rejoins the network once the bus is stable, avoiding a dealership visit for a manual reset.

CAN Mailboxes

In the context of the CAN (Controller Area Network) protocol, a mailbox refers to a storage buffer or memory location within a CAN controller that is used to send or receive messages. Mailboxes play a crucial role in both transmission and reception processes in CAN communication. Here's how they work:

1. **Transmission with Mailboxes:**

- **Message Buffering:** When a node wants to transmit a message onto the CAN bus, it writes the message data, including the identifier and data bytes, into a transmit mailbox within its CAN controller.
- **Arbitration and Transmission:** The CAN controller then initiates the transmission process by attempting to access the bus. During arbitration, the CAN controller compares the identifiers of messages in the transmit mailboxes to determine the priority of transmission.
- **Sending the Message:** If the node wins arbitration, the message stored in the highest-priority transmit mailbox is transmitted onto the bus. The CAN controller manages the timing and framing of the message transmission, including handling collisions and retries if necessary.

2. Reception with Mailboxes:

- **Message Filtering:** When a node receives a message from the CAN bus, the CAN controller checks the message identifier against a set of receive filters configured for each receive mailbox. These filters determine which messages are accepted and stored in the receive mailboxes.
- **Storing Received Messages:** If a received message matches one of the configured filters, the CAN controller stores the message data, including the identifier and data bytes, in the corresponding receive mailbox.
- **Notification and Retrieval:** The node's software can then access the receive mailboxes to retrieve and process the received messages. The CAN controller may also generate interrupts or trigger software flags to notify the processor of newly received messages in the mailboxes.

3. Role in Communication:

- **Buffering and Queuing:** Mailboxes act as buffers or queues for messages waiting to be transmitted or processed. They provide temporary storage for message data within the CAN controller, allowing for efficient handling of communication tasks.
- **Priority Handling:** Mailboxes help manage the priority of messages during transmission by providing a mechanism for arbitration based on message identifiers. This ensures that higher-priority messages are transmitted before lower-priority ones, helping maintain deterministic behavior in the communication network.
- **Filtering and Segregation:** Receive mailboxes enable filtering and segregation of incoming messages based on their identifiers. This allows nodes to selectively process messages of interest while ignoring others, improving system efficiency and reducing processor overhead.